

Gallery

Le but est de créer une application pour gérer une galerie de photos, vous avez dans l'archive un projet Angular dans lequel il vous faudra compléter les différents fichiers afin que l'application fonctionne.

N'oubliez pas de regarder dans la console de votre navigateur et la console de compilation pour voir les erreurs qu'ils restent au fur et à mesure que vous avancez dans le sujet.

Plusieurs composants sont déjà créés ainsi qu'un service, **vous n'aurez pas besoin de créer de nouveaux services ni de nouveaux composants mais juste à compléter ceux existants.**

Le design ne rentre pas en compte dans la notation, ne perdez donc pas de temps à faire quelque chose de joli !

Pour le rendu, merci de déposer seulement le répertoire **src** du projet Angular

1.LoginComponent

Gallery

thomas

CONNEXION

L'utilisateur doit renseigner ici son pseudo et lors du clique sur le bouton, il est redirigé vers le composant GalleryComponent (l'application doit passer le pseudo dans l'URL du navigateur pour que GalleryComponent le récupère).

Voici les contraintes à appliquer sur le champ du formulaire:

- Le champ est requis
- Le champ doit faire au minimum 3 caractères
- Le champ ne doit contenir que des lettres ou des chiffres (on pourra utiliser le pattern suivant : [a-zA-Z0-9]+)

Si une de ces contraintes n'est pas respectée, on doit désactiver le bouton de connexion et afficher le message "Votre nom d'utilisateur ne respecte pas les contraintes"

Gallery

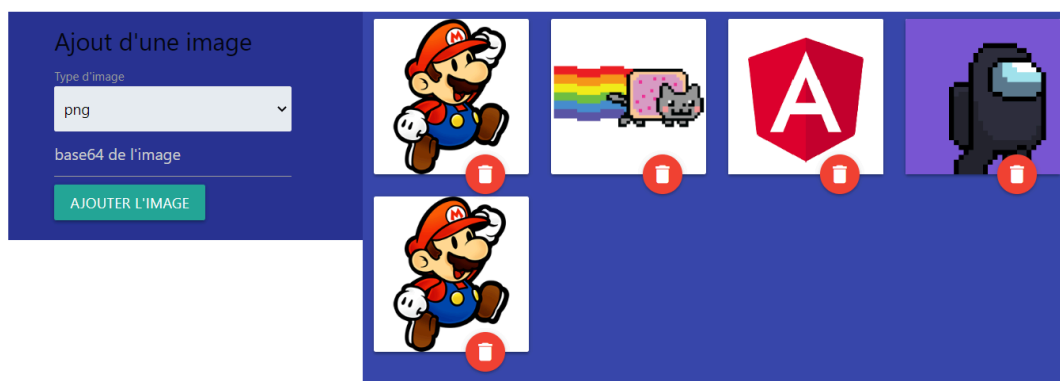
identifiant

Votre nom d'utilisateur ne respecte pas les contraintes

CONNEXION

- GalleryComponent, CarrousselComponent, NewImageFormComponent et GalleryService

Gallery



GalleryComponent sera le composant père de CarrousselComponent et NewImageFormComponent. **Il doit être le seul à utiliser le service GalleryService pour interagir avec le serveur, vous devrez utiliser la communication entre composants pour envoyer et recevoir des informations des deux composants fils.**

Le composant **NewImageFormComponent** (représentant le formulaire à gauche de la capture d'écran) contient 2 champs : le type d'image ainsi que le contenu de l'image en base64 à ajouter à la galerie de l'utilisateur.

Vous devez vous assurer que les 2 champs sont remplis pour que l'utilisateur puisse cliquer sur le bouton "ajouter image" et désactiver le bouton le cas échéant.

Le dernier composant **CarrousselComponent** sera chargé d'afficher les images de l'utilisateur, vous avez déjà des exemples pour afficher 2 images (aux formats png et gif) avec le contenu au format base64 dans le composant.

Voici une explication au cas où les exemples ne suffiraient pas :

une image est affichée dans une page grace à la balise html img, l'attribut src de cette balise contient en règle général l'url de l'image à afficher mais il est aussi possible de lui donner une image au format base64, en utilisant la syntaxe suivante :

```
<img src='data:image/png;base64,blablabla' />
```

Dans cet exemple, on peut voir que l'image est au format png et son contenu en format base64 est *blablabla*.

Si l'image est au format gif, on changera png par gif, ce qui donnerai

```
<img src='data:image/gif;base64,blablabla' />
```

Voici quelques exemples d'images au format base64 :

Image	Base64	Type d'image
	dans le fichier mario.txt ou à la fin de ce document	png
	dans le fichier cat.txt ou à la fin de ce document	gif

	dans le fichier angular.txt ou à la fin de ce document	png
	dans le fichier amongus.txt pu à la fin de ce document	gif

2. Description de l'API

Récupération des images

Méthode: GET

URL:

<https://europe-west1-cours-angular-263913.cloudfunctions.net/messenger/api/images/<NOM d'UTILISATEUR>>

Retour de l'API : GetUserImagesResponse

Création d'une nouvelle image

Méthode: POST

URL:

<https://europe-west1-cours-angular-263913.cloudfunctions.net/messenger/api/images/<NOM d'UTILISATEUR>>

Body : ImageCreation

Retour de l'API : Image

Suppression d'une image

Méthode: DELETE

URL:

<https://europe-west1-cours-angular-263913.cloudfunctions.net/messenger/api/images/<NOM d'UTILISATEUR>>

[nger/api/images/](#)<NOM d'UTILISATEUR>/<ID de l'image>

4. Model

Tous les modèles sont déjà créés dans l'application

```
export interface Image {
  id: string;
  content: string;
  type: string;
}

export interface GetUserImagesResponse {
  images: Image[];
}
```

```
export interface ImageCreation {
  content: string;
  type: string;
}
```

5. Base 64

Mario:

[illegible]

Nyan cat:

Angular :

Among us:

[illegible]

[illegible]